

Desafio de Automação Digital: Gestão de Peças, Qualidade e Armazenamento

Disciplina: Algoritmos e Lógica de Programação **Instituição:** UniFECAF

1. Contextualização do desafio

Em uma linha de montagem industrial, a inspeção de qualidade decide o destino de cada peça produzida: ela segue para o estoque/expedição ou é descartada/retrabalhada. Quando essa inspeção é feita manualmente, três problemas se acumulam com o volume de produção:

1. **Atraso operacional** — um inspetor humano tem um limite físico de quantas peças consegue medir e classificar por hora, criando um gargalo antes mesmo de a peça chegar à embalagem.
2. **Falhas de conferência** — critérios aplicados "de olho" (peso aproximado, cor a distância, comprimento estimado) variam de inspetor para inspetor e de turno para turno, gerando inconsistência na qualidade que chega ao cliente final.
3. **Custo de operação** — cada peça reprovada tarde demais no processo (por exemplo, já embalada) custa mais para corrigir do que uma peça identificada e desviada no momento da produção.

A automação digital do controle de qualidade resolve os três problemas ao mesmo tempo: ela aplica os critérios de forma **determinística** (a mesma peça sempre recebe o mesmo veredito, em qualquer turno), **instantânea** (a decisão acontece no momento do cadastro, sem fila de inspeção) e **rastreável** (cada reprovação fica registrada com o motivo exato, permitindo à fábrica identificar padrões — por exemplo, se um fornecedor de matéria-prima está entregando peças fora do peso combinado).

Este projeto prototipa, em Python, o "cérebro de decisão" desse sistema: um programa que recebe os dados de cada peça, aplica as regras de qualidade da empresa, organiza automaticamente as peças aprovadas em caixas de expedição e produz relatórios gerenciais consolidados.

2. Estruturação do raciocínio lógico

O problema foi decomposto em quatro sub-problemas menores, cada um resolvido com uma estrutura de programação específica:

2.1. Decisão: a peça é aprovada ou reprovada?

Cada peça precisa ser avaliada contra três critérios **independentes**: peso (95g–105g), cor (azul ou verde) e comprimento (10cm–20cm). A decisão de projeto aqui foi não usar uma cadeia de `if/elif/else` (que para no primeiro critério que falhar), e sim três blocos `if` separados que avaliam cada critério isoladamente e acumulam os motivos de falha em uma lista. Essa escolha importa porque uma peça pode falhar em mais de um critério ao mesmo tempo, e o relatório de qualidade precisa saber de **todos** eles, não só do primeiro encontrado — é a diferença entre um sistema que diz "essa peça está errada" e um que diz "essa peça está errada porque o peso e o comprimento estão fora do padrão".

```
if not (PESO_MIN_G <= self.peso <= PESO_MAX_G):
    motivos.append("Peso fora do padrão...")
if self.cor.strip().lower() not in CORES_VALIDAS:
    motivos.append("Cor não aceita...")
if not (COMPRIMENTO_MIN_CM <= self.comprimento <= COMPRIMENTO_MAX_CM):
    motivos.append("Comprimento fora do padrão...")

status = "Reprovada" if motivos else "Aprovada"
```

2.2. Repetição: organizar peças em lotes de tamanho fixo

O armazenamento em caixas de 10 peças é, no fundo, um problema clássico de "encher um contêiner de capacidade fixa e trocar de contêiner quando ele enche". A solução usa um laço implícito no fluxo de cadastro: toda vez que uma peça aprovada chega, ela é adicionada à caixa corrente; se essa adição faz a caixa atingir 10 peças, a caixa é marcada como fechada, arquivada na lista de caixas fechadas, e uma nova caixa vazia é criada automaticamente para receber a próxima peça aprovada. Isso elimina qualquer necessidade de o operador "abrir uma caixa nova manualmente" — o sistema decide isso por conta própria, sempre que necessário.

2.3. Funções: isolar cada responsabilidade

Em vez de um único script sequencial, o sistema foi dividido em funções e métodos pequenos, cada um com uma única responsabilidade: `avaliar()` só decide aprovação, `_alocar_em_caixa()` só decide em qual caixa a peça vai, `gerar_relatorio()` só agrega números. Essa divisão foi o que permitiu escrever 19 testes automatizados isolados — cada função pode ser testada sozinha, sem precisar simular o programa inteiro rodando.

2.4. Tratamento de exceções: o sistema nunca quebra sozinho

Situações de erro esperadas (cadastrar um ID que já existe, remover uma peça inexistente, remover uma peça que já está numa caixa fechada) foram modeladas

como exceções customizadas (`IdDuplicadoError`, `PecaNaoEncontradaError`, `RemocaoInvalidaError`), capturadas na camada de menu e convertidas em mensagens claras para o usuário — o programa continua rodando normalmente depois de um erro de entrada, em vez de travar.

3. Benefícios percebidos na solução

- **Consistência total:** a mesma peça, cadastrada duas vezes com os mesmos dados, sempre recebe o mesmo veredito — elimina a variação humana.
- **Rastreabilidade de causa:** o relatório final não diz apenas "quatro peças reprovadas", ele quebra por motivo (peso, cor, comprimento), permitindo à fábrica identificar se o problema é sistemático (por exemplo, um lote inteiro chegando com peso baixo aponta para um problema no fornecedor ou na máquina, não para peças isoladas).
- **Persistência de dados:** diferente de uma simulação que perde tudo ao fechar o programa, os dados são salvos em disco (`dados_producao.json`) a cada operação — o sistema pode ser fechado e reaberto sem perder o histórico de produção do dia.
- **Extensibilidade:** como as regras de qualidade e a capacidade da caixa são constantes isoladas no topo do arquivo `models.py` , ajustar os critérios da empresa (por exemplo, aceitar também a cor "vermelho", ou usar caixas de 20 peças) não exige reescrever a lógica do sistema.

4. Desafios enfrentados no desenvolvimento

- **Decidir a granularidade da avaliação:** a tentação inicial era usar `if/elif` para os três critérios, o que pareceria mais "simples" — mas isso escondia informação do relatório (só o primeiro motivo apareceria). A solução foi entender que "aprovado/reprovado" é uma decisão, mas "por quê" é uma **coleção** de decisões independentes.
- **Regra de remoção em caixas fechadas:** decidir se uma peça poderia ser removida depois de alocada em uma caixa já fechada exigiu pensar no problema do ponto de vista do processo real de fábrica — uma caixa fechada já está pronta para expedição, então permitir remoção depois quebraria a integridade do lote. Essa regra de negócio não estava explícita no desafio original; foi uma decisão de design justificada pela lógica do próprio domínio.
- **Numeração de caixas após reinício do programa:** como o sistema persiste dados entre execuções, foi preciso garantir que a numeração de caixas continuasse de onde parou (e não reiniciasse do #1), o que exigiu salvar e recarregar também o contador interno de caixas, não apenas as peças.

5. Reflexão final: expansão para um cenário real

Este protótipo resolve o problema no nível lógico — a decisão de aprovação e a organização em lotes —, mas uma implantação real em uma fábrica passaria por três frentes de evolução, descritas em detalhe no `README.md` do repositório:

1. **Sensores físicos** substituindo a digitação manual dos dados (balança digital, sensor de cor, sensor de distância a laser), conectados por um microcontrolador que alimenta o sistema em tempo real.
2. **Inteligência artificial**, especialmente visão computacional, para detectar defeitos que não se reduzem a peso/cor/comprimento (rachaduras, rebarbas, deformações), e modelos preditivos que usam o histórico de reprovações para antecipar falhas na linha.
3. **Integração industrial**, transformando o programa de terminal em um serviço conectado a um sistema MES/ERP da fábrica, com dashboards em tempo real para os gestores de produção.

O núcleo de decisão construído aqui — avaliar critérios, alocar em lotes, consolidar relatórios — é exatamente o tipo de lógica que continuaria no centro de qualquer uma dessas evoluções; o protótipo em Python é, nesse sentido, o esqueleto lógico de um sistema industrial real.